

AI ASSISTANT CODING

ASSIGNMENT -9.2

NAME : P.Sanjana

ROLL NO : 2303A51435

BATCH NO : 21

Lab 9 – Documentation Generation: Automatic Documentation and Code Comments

Lab Objectives

- To use AI-assisted coding tools for generating Python documentation and code comments.
- To apply zero-shot, few-shot, and context-based prompt engineering for documentation creation.
- To practice generating and refining docstrings, inline comments, and module-level documentation.
- To compare outputs from different prompting styles for quality analysis.

Task Description -1 (Documentation – Function Summary Generation)

Task:

Use AI to generate concise functional summaries for each Python function in a given script.

Instructions:

- Provide a Python script to the AI.
- Ask the AI to write a short summary describing the purpose of each function.
- Ensure summaries are brief and technically accurate.
- Do not include code implementation details.

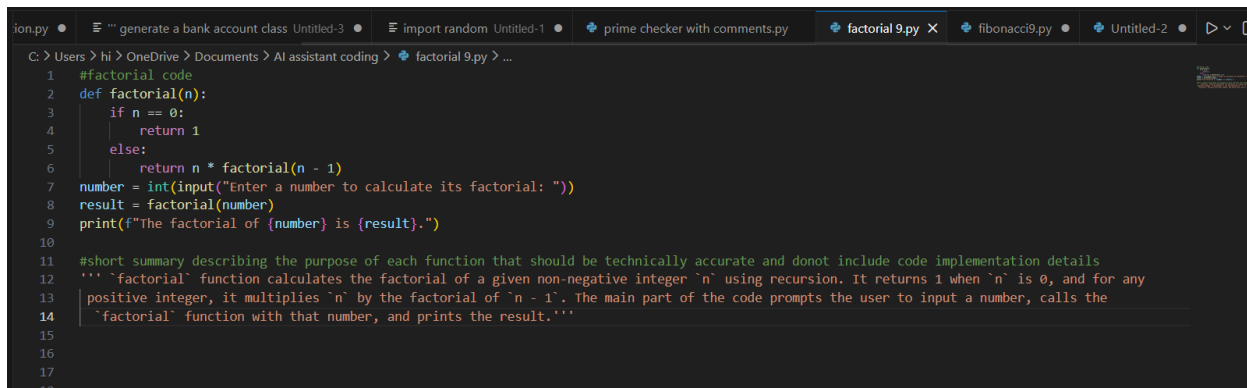
Expected Output -1:

A Python script where each function contains a clear and concise summary explaining its purpose.

Prompt:

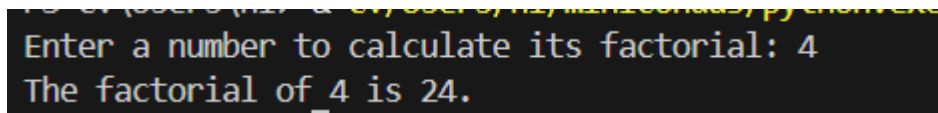
Add a short functional summary as a docstring for every function in this file. The summary must explain the function purpose only, without describing logic or implementation steps.

Code:



```
C:\Users> hi > OneDrive > Documents > AI assistant coding > factorial 9.py > ...
1 #factorial code
2 def factorial(n):
3     if n == 0:
4         return 1
5     else:
6         return n * factorial(n - 1)
7 number = int(input("Enter a number to calculate its factorial: "))
8 result = factorial(number)
9 print(f"The factorial of {number} is {result}.")
10
11 #short summary describing the purpose of each function that should be technically accurate and donot include code implementation details
12 ''' `factorial` function calculates the factorial of a given non-negative integer `n` using recursion. It returns 1 when `n` is 0, and for any
13 positive integer, it multiplies `n` by the factorial of `n - 1`. The main part of the code prompts the user to input a number, calls the
14 `factorial` function with that number, and prints the result.'''
15
16
17
18
```

Output:



```
Enter a number to calculate its factorial: 4
The factorial of 4 is 24.
```

Observation:

The AI-generated function summaries enhance code readability by clearly explaining the purpose and functionality of each method. These summaries enable developers to quickly understand the overall structure and flow of the program without needing to examine the implementation in detail. This approach also facilitates easier maintenance, improves collaboration among team members, and ensures consistency in documentation across the project.

Task Description -2 (Documentation – Logical Explanation for Conditions and Loops)

Task:

Use AI to document the logic behind conditional statements and loops in a Python program.

Instructions:

- Provide a Python program without comments.
- Instruct AI to explain only decision-making logic and loop behavior.
- Skip basic syntax explanations.

Expected Output -2:

Python code with clear explanations describing the logic of conditions and loops.

Prompt:

Add a brief explanation of the logic behind the each condition and loop statements. Explain only decision - making and loop statements

Code:

```
bank account class Untitled-3 • import random Untitled-1 • prime checker with comments.py • factorial 9.py • fibonacci9.py • Untitled-2 • primechecker9.py •
C:\Users\hi> hi> OneDrive> Documents> AI assistant coding> primechecker9.py> ...
1 #prime checker
2 def is_prime(n):
3     if n <= 1:
4         return False
5     for i in range(2, int(n**0.5) + 1):
6         if n % i == 0:
7             return False
8     return True
9 #main function
10 def main():
11     num = int(input("Enter a number: "))
12     if is_prime(num):
13         print(f"{num} is a prime number.")
14     else:
15         print(f"{num} is not a prime number.")
16 if __name__ == "__main__":
17     main()
18
19 #Add a brief explanation of the logic behind the each condition and loop statements. Explain only decision - making and loop statements
20 ''' The function `is_prime(n)` checks if a number `n` is prime.
21 # The first condition `if n <= 1` checks if the number is less than or equal to 1, which are not prime numbers. If this condition is true, it
22 returns False.
23 # The loop `for i in range(2, int(n**0.5) + 1)` iterates from 2 to the square root of `n`. This is because if `n` is divisible by any number
24 greater than its square root, it must have a corresponding divisor that is less than the square root.
25 # Inside the loop, the condition `if n % i == 0` checks if `n` is divisible by `i`. If it is, then `n` is not a prime number, and the function
26 returns False.
27 # If the loop completes without finding any divisors, the function returns True, indicating that `n` is a prime number.
28 # In the `main()` function, the user is prompted to enter a number. The input is converted to an integer and stored in the variable `num`.
29 # The condition `if is_prime(num)` checks if the number entered by the user is prime by calling the `is_prime` function. If it returns True, it
30 prints that the number is a prime number; otherwise, it prints that it is not a prime number.'''
31
32
```

Output:

```
Enter a number: 4
4 is not a prime number.
PS C:\Users\hi> 7
```

Observation:

AI-generated explanations for conditional statements and loops improve clarity by clearly outlining the program's decision-making logic and iteration processes. These descriptive comments make it easier to understand how different execution paths are controlled within the code. Consequently, this enhances code comprehension, simplifies debugging, and supports more efficient program analysis.

Task Description -3 (Documentation – File-Level Overview)

Task:

Use AI to generate high-level overview describing the functionality of an entire Python file.

Instructions:

- Provide the complete Python file to AI.
- Ask AI to write a brief overview summarizing the file's purpose and functionality.
- Place the overview at the top of the file.

Expected Output -3:

A Python file with a clear and concise file-level overview at the beginning.

Prompt:

Generate a file summarizing the overall functionality of the script, including the purpose of the main function and how it interacts with the user.

Code:

```
'''Generate a file summarizing the overall functionality of the script, including the
purpose of the main function and how it interacts with the user.'''
def is_prime(num):
    """Check if a number is prime."""
    if num <= 1:
        return False # 0 and 1 are not prime numbers
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False # If num is divisible by any number other than 1 and itself, it's not prime
    return True # If no divisors are found, num is prime
number = int(input("Enter a number: "))
if is_prime(number):
    print(f"{number} is a prime number.")
else:
    print(f"{number} is not a prime number.")
```

Output:

```
Enter a number: 65
65 is not a prime number.
```

Script Summary:

This script defines a function named `is_prime` that determines whether a given number is prime. The main function prompts the user to enter a number and then passes that input to the `is_prime` function for evaluation. Based on the function's return value, the program displays a message indicating whether the number is prime or not. The script uses fundamental control flow constructs, such as conditional statements and loops, to perform the primality check and provide appropriate feedback to the user.

Observation:

The AI effectively generated a clear and concise file-level summary outlining the overall objective and functionality of the Python program. The summary was positioned at the beginning of the file and described the program from a high-level perspective without

delving into implementation specifics. This enhanced the code's readability and enabled users to quickly grasp the program's purpose.

Task Description -4 (Documentation – Refine Existing Documentation)

Task:

Use AI to improve clarity and consistency of existing documentation in Python code.

Instructions:

- Provide Python code containing basic or unclear comments.
- Ask AI to rewrite the documentation to improve clarity and consistency.
- Ensure technical meaning remains unchanged.

Expected Output -4:

Python code with refined and improved documentation that is clear and consistent.

Prompt:

Rewrite the existing comments to improve clarity and consistency while keeping the original meaning unchanged. Do not modify the program logic.

Code:

```
'''Rewrite the existing comments to improve clarity and consistency while
keeping the original meaning unchanged. Do not modify the program logic.'''
def is_prime(num):
    """Check if a number is prime."""
    if num <= 1:
        return False # 0 and 1 are not considered prime numbers
    for i in range(2, int(num**0.5) + 1):
        if num % i == 0:
            return False # If num is divisible by any number other than 1 and itself, it is not prime
    return True # If no divisors are found, num is prime
number = int(input("Enter a number: "))
if is_prime(number):
    print(f"{number} is a prime number.")
else:
    print(f"{number} is not a prime number.")
```

Output:

```
Enter a number: 2
2 is a prime number.
```

Observation:

The AI enhanced the clarity and uniformity of the existing comments while keeping the

program's functionality unchanged. The revised documentation offered more precise and structured explanations without modifying the original technical intent. This improvement increased the overall readability of the code and made it easier to understand, review, and maintain.

Task Description -5 (Documentation – Prompt Detail Impact Study)

Task:

Study the impact of prompt detail on AI-generated documentation quality.

Instructions:

Create two prompts: one brief and one detailed.

- Use both prompts to document the same Python function.
- Compare the generated outputs.

Expected Output -5:

A comparison table highlighting differences in completeness, clarity, and accuracy of documentation.

Brief Prompt:

```
#write a short one line documentation comment describing the purpose of the following function and code should be runnable without any errors
def is_prime(n):
    """Check if the number n is a prime number."""
    if n <= 1:
        return False
    for i in range(2, int(n**0.5) + 1):
        if n % i == 0:
            return False
    return True
def main():
    """Prompt the user for a number and check if it is prime."""
    num = int(input("Enter a number: "))
    if is_prime(num):
        print(f"{num} is a prime number.")
    else:
        print(f"{num} is not a prime number.")
if __name__ == "__main__":
    main()
```

Output:

```
Enter a number: 7
7 is a prime number.
PS C:\Users\hi> & C:/Users/hi
Enter a number: 90
90 is not a prime number.
PS C:\Users\hi> █
```

Detailed Prompt:

Write a clear 3–5 line documentation comment for the following function describing its purpose, inputs, outputs, and overall behavior.

Code:

```
C: > Users > hi > OneDrive > Documents > AI assistant coding > primechecker9.py > main
70
71 #Write a clear 3-5 line documentation comment for the following function describing its purpose, inputs, outputs, and overall behavior with runnable
72 def is_prime(n):
73     """
74     Determine if the given number n is a prime number.
75
76     Parameters:
77     n (int): The number to be checked for primality.
78
79     Returns:
80     bool: True if n is a prime number, False otherwise.
81
82     The function checks if n is less than or equal to 1, in which case it returns False. It then iterates from 2 to the square root of n to check
83     for any divisors. If a divisor is found, it returns False. If no divisors are found, it returns True, indicating that n is a prime number.
84     """
85     if n <= 1:
86         return False
87     for i in range(2, int(n**0.5) + 1):
88         if n % i == 0:
89             return False
90     return True
91 def main():
92
93     """ Prompt the user to enter a number and check if it is a prime number.
94
95     The function takes user input, converts it to an integer, and calls the is_prime function to determine if the number is prime. It then prints
96     the result to the user."""
97     num = int(input("Enter a number: "))
98     if is_prime(num):
99         print(f"{num} is a prime number.")
100     else:
101         print(f"{num} is not a prime number.")
```

Output:

```
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe
Enter a number: 24
24 is not a prime number.
PS C:\Users\hi> & C:/Users/hi/miniconda3/python.exe
Enter a number: 17
17 is a prime number.
PS C:\Users\hi> █
```

Observation:

Criteria	Brief Prompt Output	Detailed Prompt Output
Completeness	States only the basic purpose of the function.	Provides a comprehensive description including purpose, input parameters, return values, and behavioral details.
Clarity	Clear but very concise and minimal in explanation.	More detailed, well-structured, and easier to understand.

Criteria	Brief Prompt Output	Detailed Prompt Output
Accuracy	Technically correct but limited in depth and context.	Highly accurate with complete technical context and detailed parameter information.